

Part 1. Tables

Here is our project in GCP with a SQL instance supporting our DB.

The screenshot shows the Google Cloud console interface for a project named 'FocusFeed-93'. The 'Connections' section displays the 'focusfeed-db' instance, which is a MySQL 8.0 database. The 'Networking' tab is selected, showing the connection name 'focusfeed-93:us-central1:focusfeed-db'.

Below the console, a terminal window is open, showing the output of several SQL queries. The queries describe the schema of three tables: 'Keywords', 'FeedNotes', and 'Interactions'.

```
mysql> DESCRIBE Keywords;
+-----+-----+-----+-----+-----+-----+
| keywordId | int | NO | PRI | NULL | |
+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> DESCRIBE FeedNotes;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| noteId | int | NO | PRI | NULL | auto_increment |
| userId | int | YES | | NULL | |
| articleId | int | YES | MUL | NULL | |
| title | varchar(255) | YES | | NULL | |
| summary | varchar(300) | YES | | NULL | |
| timestamp | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)

mysql> DESCRIBE Interactions;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| interactionId | int | NO | PRI | NULL | auto_increment |
| userId | int | YES | | NULL | |
| noteId | int | YES | | NULL | |
| timestamp | datetime | YES | | NULL | |
| interactionType | enum('click','like','dislike','favorite','unfavorite') | YES | | NULL | |
| interactionDuration | double | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)

mysql>
```

Five main tables:

```
mysql> DESCRIBE Articles;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| articleId | int | NO | PRI | NULL | auto_increment |
| title | varchar(255) | YES | | NULL | |
| abstract | varchar(255) | YES | | NULL | |
| url | varchar(255) | YES | | NULL | |
| publishedAt | datetime | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
5 rows in set (0.01 sec)

mysql> DESCRIBE articlesKeywords;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| articleId | int | NO | PRI | NULL | |
| keywordId | int | NO | PRI | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> DESCRIBE Keywords;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| keywordId | int | NO | PRI | NULL | auto_increment |
| keyword | varchar(200) | YES | UNI | NULL | |
+-----+-----+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> DESCRIBE FeedNotes;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| noteId | int | NO | PRI | NULL | auto_increment |
| userId | int | YES | | NULL | |
| articleId | int | YES | MUL | NULL | |
| title | varchar(255) | YES | | NULL | |
| summary | varchar(300) | YES | | NULL | |
| timestamp | timestamp | YES | | CURRENT_TIMESTAMP | DEFAULT_GENERATED |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)

mysql> DESCRIBE Interactions;
+-----+-----+-----+-----+-----+-----+
| Field | Type | Null | Key | Default | Extra |
+-----+-----+-----+-----+-----+-----+
| interactionId | int | NO | PRI | NULL | auto_increment |
| userId | int | YES | | NULL | |
| noteId | int | YES | | NULL | |
| timestamp | datetime | YES | | NULL | |
| interactionType | enum('click','like','dislike','favorite','unfavorite') | YES | | NULL | |
| interactionDuration | double | YES | | NULL | |
+-----+-----+-----+-----+-----+-----+
6 rows in set (0.01 sec)
```

Commands to create tables:

```
CREATE TABLE Articles (
    articleId INT PRIMARY KEY AUTO_INCREMENT,
    title VARCHAR(255),
    abstract VARCHAR(255),
    url VARCHAR(255),
    publishedAt DATETIME
);

CREATE TABLE articlesKeywords (
    articleId INT REFERENCES Articles(articleId),
    keywordId INT REFERENCES Keywords(keywordId),
    PRIMARY KEY (articleId, keywordId)
);

CREATE TABLE Keywords (
    keywordId INT PRIMARY KEY AUTO_INCREMENT,
```

```

        keyword VARCHAR(200) UNIQUE -- normalized keywords to help semantic
search and indexing
);
CREATE TABLE FeedNotes (
    noteId INT AUTO_INCREMENT PRIMARY KEY,
    userId INT,
    articleId INT,
    FOREIGN KEY (articleId) REFERENCES Articles(articleId),
    title VARCHAR(255),
    summary VARCHAR(300),
    timestamp TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE Interactions (
    interactionId INT PRIMARY KEY AUTO_INCREMENT,
    userId INT REFERENCES FeedNotes(userId),
    noteId INT REFERENCES FeedNotes(noteId),
    timestamp DATETIME,
    interactionType ENUM('click', 'like', 'dislike', 'favorite',
'unfavorite'),
    interactionDuration DOUBLE
);

```

Insert data count exceeds 1000 for four tables.

```

Database changed
mysql> SELECT COUNT(*) FROM Articles;
+-----+
| COUNT(*) |
+-----+
|      1266 |
+-----+
1 row in set (0.03 sec)

mysql> SELECT COUNT(*) FROM Keywords;
+-----+
| COUNT(*) |
+-----+
|      2772 |
+-----+
1 row in set (0.01 sec)

mysql> SELECT COUNT(*) FROM articlesKeywords;
+-----+
| COUNT(*) |
+-----+
|      5896 |
+-----+
1 row in set (0.01 sec)

mysql> SELECT COUNT(*) FROM FeedNotes;
+-----+
| COUNT(*) |
+-----+
|      1266 |
+-----+
1 row in set (0.00 sec)

mysql>

```

Part 2, Indexing

Advanced Query #1: Articles with the most amount of associated FeedNotes, then most amount of associated keywords

```

SELECT a.articleId, a.title, COUNT(DISTINCT f.noteId) AS num_notes,
COUNT(DISTINCT k.keywordId) AS num_keywords
FROM Articles a
JOIN FeedNotes f ON a.articleId = f.articleId
JOIN articlesKeywords ak ON a.articleId = ak.articleId

```

```
JOIN Keywords k ON ak.keywordId = k.keywordId
GROUP BY a.articleId, a.title
ORDER BY num_notes DESC, num_keywords DESC
LIMIT 15;
```

Returns:

```
mysql> SELECT a.articleId, a.title, COUNT(DISTINCT f.noteId) AS num_notes, COUNT(DISTINCT k.keywordId) AS num_keywords
-> FROM Articles a
-> JOIN FeedNotes f ON a.articleId = f.articleId
-> JOIN articlesKeywords ak ON a.articleId = ak.articleId
-> JOIN Keywords k ON ak.keywordId = k.keywordId
-> GROUP BY a.articleId, a.title
-> ORDER BY num_notes DESC, num_keywords DESC
-> LIMIT 15;
```

articleId	title	num_notes	num_keywords
28	Alan Moore Beckons Readers Into a Rich New Fantasy World	11	7
123	Europe Tries to Boost Military Spending as America Realigns	6	14
122	A Tariff Pile-On Threatens to Escalate a Global Trade War	2	10
308	52 Places to Go in 2025	1	57
41	Ladies and Gentlemen, 38 Key Musical Moments on 'S.N.L.'	1	48
1255	Long Before She Was Charged With Murder, a Group of Men Had Raised an Alarm	1	27
520	How to Watch the Golden Globes 2025: Date, Time and Streaming	1	27
1179	The Highbrow and Lowbrow of the Trump Era	1	24
121	Europe Welcomes a Ukraine Cease-Fire Offer and a Revival of U.S. Aid	1	23
1102	'A Different Man' and 'Sing Sing' Rule the Gotham Awards	1	22
38	Celebrities Start Partying Before the Oscars and Keep Going All Weekend	1	22
636	The Playlist: George Clinton Reunites With Parliament, and 13 More New Songs	1	22
635	The Playlist: Esperanza Spalding's Intoxicating Groove, and 12 More New Songs	1	21
127	Meeting Again in Paris, European Leaders Try to Recalibrate After Trump Sides With Russia	1	21
160	10 Great Thrillers Set in Remote, Hard-to-Reach Places	1	20

15 rows in set (0.04 sec)

Advanced queries:

1. Index articlesKeywords(keywordId) no speedup:

```

-> Limit: 15 row(s) (actual time=31.5..31.6 rows=15 loops=1)
-> Sort: num_notes DESC, num_keywords DESC, limit input to 15 row(s) per chunk (actual time=31.5..31.5 rows=15 loops=1)
-> Stream results (actual time=23.4..31.3 rows=860 loops=1)
-> Group aggregate: count(distinct FeedNotes.noteId), count(distinct Keywords.keywordId) (actual time=23.4..30.6 rows=860 loops=1)
-> Sort: a.articleId, a.title (actual time=23.2..24.1 rows=6046 loops=1)
-> Stream results (cost=4504 rows=8109) (actual time=0.193..16.9 rows=6046 loops=1)
-> Nested loop inner join (cost=4504 rows=8109) (actual time=0.186..14.5 rows=6046 loops=1)
-> Nested loop inner join (cost=1666 rows=8109) (actual time=0.174..7.98 rows=6046 loops=1)
-> Nested loop inner join (cost=554 rows=1191) (actual time=0.114..2.84 rows=1282 loops=1)
-> Filter: (f.articleId is not null) (cost=137 rows=1191) (actual time=0.0792..0.539 rows=1282 loops=1)
-> Covering index scan on f using articleId (cost=137 rows=1191) (actual time=0.0714..0.416 rows=1282 loops=1)
-> Single-row index lookup on a using PRIMARY (articleId=f.articleId) (cost=0.25 rows=1) (actual time=0.00157..0.0016 rows=1 loops=1282)
-> Covering index lookup on ak using PRIMARY (articleId=f.articleId) (cost=0.253 rows=6.81) (actual time=0.00241..0.00357 rows=4.72 loops=1282)
-> Single-row covering index lookup on k using PRIMARY (keywordId=ak.keywordId) (cost=0.25 rows=1) (actual time=862e-6..891e-6 rows=1 loops=6046)

1 row in set (0.06 sec)

mysql> CREATE INDEX idx ON articlesKeywords(keywordId);
Query OK, 0 rows affected (0.15 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> EXPLAIN ANALYZE SELECT a.articleId, a.title, COUNT(DISTINCT f.noteId) AS num_notes, COUNT(DISTINCT k.keywordId) AS num_keywords FROM Articles a JOIN FeedNotes f ON a.articleId=f.articleId
DESC, num_keywords DESC LIMIT 15;
```

```

EXPLAIN
+-----+
| EXPLAIN
+-----+
-> Limit: 15 row(s) (actual time=31.9..31.9 rows=15 loops=1)
-> Sort: num_notes DESC, num_keywords DESC, limit input to 15 row(s) per chunk (actual time=31.9..31.9 rows=15 loops=1)
-> Stream results (actual time=24.6..31.7 rows=860 loops=1)
-> Group aggregate: count(distinct FeedNotes.noteId), count(distinct Keywords.keywordId) (actual time=24.6..31.4 rows=860 loops=1)
-> Sort: a.articleId, a.title (actual time=24.5..25.1 rows=6046 loops=1)
-> Stream results (cost=4504 rows=8109) (actual time=0.143..17.5 rows=6046 loops=1)
-> Nested loop inner join (cost=4504 rows=8109) (actual time=0.139..15.1 rows=6046 loops=1)
-> Nested loop inner join (cost=1666 rows=8109) (actual time=0.132..8.3 rows=6046 loops=1)
-> Nested loop inner join (cost=554 rows=1191) (actual time=0.0794..2.86 rows=1282 loops=1)
-> Filter: (f.articleId is not null) (cost=137 rows=1191) (actual time=0.0574..0.54 rows=1282 loops=1)
-> Covering index scan on f using articleId (cost=137 rows=1191) (actual time=0.0559..0.42 rows=1282 loops=1)
-> Single-row index lookup on a using PRIMARY (articleId=f.articleId) (cost=0.25 rows=1) (actual time=0.00159..0.00162 rows=1 loops=1282)
-> Covering index lookup on ak using PRIMARY (articleId=f.articleId) (cost=0.253 rows=6.81) (actual time=0.00261..0.00378 rows=4.72 loops=1282)

```

2. Index Articles(title) slight speedup, maybe negligible (loop structure looks the same)

```

+-----+
-> Limit: 15 row(s) (actual time=37.7..37.7 rows=15 loops=1)
-> Sort: num_notes DESC, num_keywords DESC, limit input to 15 row(s) per chunk (actual time=37.7..37.7 rows=15 loops=1)
-> Stream results (actual time=30.8..37.5 rows=860 loops=1)
-> Group aggregate: count(distinct FeedNotes.noteId), count(distinct Keywords.keywordId) (actual time=30.8..37.2 rows=860 loops=1)
-> Sort: a.articleId, a.title (actual time=30.6..31.3 rows=6046 loops=1)
-> Stream results (cost=4444 rows=8000) (actual time=0.507..22.1 rows=6046 loops=1)
-> Nested loop inner join (cost=4444 rows=8000) (actual time=0.498..19.5 rows=6046 loops=1)
-> Nested loop inner join (cost=1644 rows=8000) (actual time=0.469..10.7 rows=6046 loops=1)
-> Nested loop inner join (cost=547 rows=1175) (actual time=0.191..3.9 rows=1282 loops=1)
-> Filter: (f.articleId is not null) (cost=136 rows=1175) (actual time=0.123..0.516 rows=1282 loops=1)
-> Covering index scan on f using articleId (cost=136 rows=1175) (actual time=0.12..0.685 rows=1282 loops=1)
-> Single-row index lookup on a using PRIMARY (articleId=f.articleId) (cost=0.25 rows=1) (actual time=0.00207..0.0021 rows=1 loops=1282)
-> Covering index lookup on ak using PRIMARY (articleId=f.articleId) (cost=0.253 rows=6.81) (actual time=0.00346..0.00487 rows=4.72 loops=1282)
-> Single-row covering index lookup on k using PRIMARY (keywordId=ak.keywordId) (cost=0.25 rows=1) (actual time=0.00123..0.00126 rows=1 loops=6046)
|

+-----+
1 row in set (0.07 sec)

mysql> CREATE INDEX idx ON Articles(title);
Query OK, 0 rows affected (0.13 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> EXPLAIN ANALYZE SELECT a.articleId, a.title, COUNT(DISTINCT f.noteId) AS num_notes, COUNT(DISTINCT k.keywordId) AS num_keywords FROM Articles a JOIN FeedNotes f ON a.articleId=f.noteId JOIN Keywords k ON a.articleId=k.keywordId DESC, num_keywords DESC LIMIT 15;

+-----+
EXPLAIN
+-----+

+-----+
-> Limit: 15 row(s) (actual time=31..31 rows=15 loops=1)
-> Sort: num_notes DESC, num_keywords DESC, limit input to 15 row(s) per chunk (actual time=31..31 rows=15 loops=1)
-> Stream results (actual time=24..30.8 rows=860 loops=1)
-> Group aggregate: count(distinct FeedNotes.noteId), count(distinct Keywords.keywordId) (actual time=24..30.5 rows=860 loops=1)
-> Sort: a.articleId, a.title (actual time=23.9..24.5 rows=6046 loops=1)
-> Stream results (cost=4444 rows=8000) (actual time=0.125..17.1 rows=6046 loops=1)
-> Nested loop inner join (cost=4444 rows=8000) (actual time=0.122..14.6 rows=6046 loops=1)
-> Nested loop inner join (cost=1644 rows=8000) (actual time=0.116..8.27 rows=6046 loops=1)
-> Nested loop inner join (cost=547 rows=1175) (actual time=0.059..2.87 rows=1282 loops=1)
-> Filter: (f.articleId is not null) (cost=136 rows=1175) (actual time=0.0431..0.518 rows=1282 loops=1)
-> Covering index scan on f using articleId (cost=136 rows=1175) (actual time=0.0419..0.392 rows=1282 loops=1)
-> Single-row index lookup on a using PRIMARY (articleId=f.articleId) (cost=0.25 rows=1) (actual time=0.00159..0.00163 rows=1 loops=1282)
-> Covering index lookup on ak using PRIMARY (articleId=f.articleId) (cost=0.253 rows=6.81) (actual time=0.00261..0.00377 rows=4.72 loops=1282)
-> Single-row covering index lookup on k using PRIMARY (keywordId=ak.keywordId) (cost=0.25 rows=1) (actual time=0.00123..0.00126 rows=1 loops=6046)
|

```

3. Index Articles(publishedAt, same as Articles(title)

```

+-----+
-> Limit: 15 row(s) (actual time=30.1..30.1 rows=15 loops=1)
-> Sort: num_notes DESC, num_keywords DESC, limit input to 15 row(s) per chunk (actual time=30.1..30.1 rows=15 loops=1)
-> Stream results (actual time=23.5..29.9 rows=860 loops=1)
-> Group aggregate: count(distinct FeedNotes.noteId), count(distinct Keywords.keywordId) (actual time=23.4..29.6 rows=860 loops=1)
-> Sort: a.articleId, a.title (actual time=23.3..23.9 rows=6046 loops=1)
-> Stream results (cost=4444 rows=8000) (actual time=0.119..16.3 rows=6046 loops=1)
-> Nested loop inner join (cost=4444 rows=8000) (actual time=0.116..14.1 rows=6046 loops=1)
-> Nested loop inner join (cost=1644 rows=8000) (actual time=0.108..7.91 rows=6046 loops=1)
-> Nested loop inner join (cost=547 rows=1175) (actual time=0.0565..2.73 rows=1282 loops=1)
-> Filter: (f.articleId is not null) (cost=136 rows=1175) (actual time=0.0398..0.541 rows=1282 loops=1)
-> Covering index scan on f using articleId (cost=136 rows=1175) (actual time=0.0382..0.416 rows=1282 loops=1)
-> Single-row index lookup on a using PRIMARY (articleId=f.articleId) (cost=0.25 rows=1) (actual time=0.00149..0.00152 rows=1 loops=1282)
-> Covering index lookup on ak using PRIMARY (articleId=f.articleId) (cost=0.253 rows=6.81) (actual time=0.0024..0.0036 rows=4.72 loops=1282)
-> Single-row covering index lookup on k using PRIMARY (keywordId=ak.keywordId) (cost=0.25 rows=1) (actual time=0.00123..0.00126 rows=1 loops=6046)
|

```

Advanced Query #2: -- Articles with keyword Germany that don't include keyword Politics

```

SELECT a.title
FROM Articles a
WHERE a.articleId IN (
    SELECT ak1.articleId
    FROM articlesKeywords ak1
    JOIN Keywords k1 ON ak1.keywordId = k1.keywordId
    WHERE k1.keyword = "Germany"
)
AND a.articleId NOT IN (
    SELECT ak2.articleId

```

```

FROM articlesKeywords ak2
JOIN Keywords k2 ON ak2.keywordId = k2.keywordId
WHERE k2.keyword = 'Politics'
)
LIMIT 15;

```

Returns:

```

mysql> SELECT a.title
-> FROM Articles a
-> WHERE a.articleId IN (
->     SELECT ak1.articleId
->     FROM articlesKeywords ak1
->     JOIN Keywords k1 ON ak1.keywordId = k1.keywordId
->     WHERE k1.keyword = 'Germany'
-> )
-> AND a.articleId NOT IN (
->     SELECT ak2.articleId
->     FROM articlesKeywords ak2
->     JOIN Keywords k2 ON ak2.keywordId = k2.keywordId
->     WHERE k2.keyword = 'Politics'
-> )
-> LIMIT 15;
+-----+
| title                                     |
+-----+
| Things Are Terrible in Europe, and They're Only Going to Get Worse |
| Europäisch werden                     |
| When Germany Votes, It Will Be About the Economy                   |
| Prince zu Loewenstein Excepts                                           |
| NAZI ENVOY TO SWEDEN; Stockholm Hears Thomsen Will Replace Prince zu Wied There |
+-----+
5 rows in set (0.01 sec)

```

Advanced Queries:

1. Index articlesKeywords(keywordId) speedup (this works because the composite key for articlesKeywords is ordered by articleId first):

```
--> )
--> LIMIT 15;

+-----+
| EXPLAIN
+-----+

+-----+
| -> Limit: 15 row(s) (cost=781 rows=15) (actual time=0.143..1.79 rows=5 loops=1)
|   -> Nested loop antijoin (cost=781 rows=537) (actual time=0.142..1.79 rows=5 loops=1)
|     -> Nested loop inner join (cost=727 rows=537) (actual time=0.128..1.78 rows=5 loops=1)
|       -> Filter: (ak1.keywordId = '226') (cost=540 rows=537) (actual time=0.0845..1.72 rows=5 loops=1)
|         -> Covering index scan on ak1 using PRIMARY (cost=540 rows=5365) (actual time=0.0319..1.36 rows=5896 loops=1)
|           -> Single-row index lookup on a using PRIMARY (articleId=ak1.articleId) (cost=0.25 rows=1) (actual time=0.00542..0.00546 rows=1 loops=5)
|     -> Nested loop inner join (cost=134 rows=1) (actual time=0.00262..0.00262 rows=0 loops=5)
|       -> Constant row from k2 (cost=652e-6..652e-6 rows=1) (actual time=0.00245..0.00245 rows=0 loops=5)
|       -> Single-row covering index lookup on ak2 using PRIMARY (articleId=ak1.articleId, keywordId=NULL) (cost=0.25 rows=1) (never executed)
|
+-----+

1 row in set (0.01 sec)

mysql> CREATE INDEX idx ON articlesKeywords.keywordId;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near '' at line 1
mysql> CREATE INDEX idx ON articlesKeywords(keywordId);
Query OK, 0 rows affected (0.20 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> EXPLAIN ANALYZE SELECT a.title FROM Articles a WHERE a.articleId IN ( SELECT ak1.articleId FROM articlesKeywords ak1 JOIN Keywords k1 ON ak1.keywordId
= k1.keywordId WHERE k2.keyword = 'Politics' ) LIMIT 15;

+-----+
| EXPLAIN
+-----+

+-----+
| -> Limit: 15 row(s) (cost=3.85 rows=5) (actual time=0.0424..0.0596 rows=5 loops=1)
|   -> Nested loop antijoin (cost=3.85 rows=5) (actual time=0.0369..0.0538 rows=5 loops=1)
|     -> Nested loop inner join (cost=3.1 rows=5) (actual time=0.0261..0.0414 rows=5 loops=1)
|       -> Covering index lookup on ak1 using idx (keywordId='226') (cost=1.35 rows=5) (actual time=0.0102..0.0137 rows=5 loops=1)
|       -> Single-row index lookup on a using PRIMARY (articleId=ak1.articleId) (cost=0.27 rows=1) (actual time=0.005..0.00505 rows=1 loops=5)
|     -> Nested loop inner join (cost=1.42 rows=1) (actual time=0.0021..0.0021 rows=0 loops=5)
|       -> Constant row from k2 (cost=0.07..0.07 rows=1) (actual time=0.00193..0.00193 rows=0 loops=5)
|       -> Single-row covering index lookup on ak2 using PRIMARY (articleId=ak1.articleId, keywordId=NULL) (cost=0.27 rows=1) (never executed)
|
+-----+

1 row in set (0.00 sec)
```

2. Index Keywords(keywordId) no speedup:

```
| EXPLAIN
+-----+

+-----+
| -> Limit: 15 row(s) (cost=781 rows=15) (actual time=0.105..1.85 rows=5 loops=1)
|   -> Nested loop antijoin (cost=781 rows=537) (actual time=0.103..1.85 rows=5 loops=1)
|     -> Nested loop inner join (cost=727 rows=537) (actual time=0.0906..1.83 rows=5 loops=1)
|       -> Filter: (ak1.keywordId = '226') (cost=540 rows=537) (actual time=0.0658..1.79 rows=5 loops=1)
|         -> Covering index scan on ak1 using PRIMARY (cost=540 rows=5365) (actual time=0.0278..1.45 rows=5896 loops=1)
|           -> Single-row index lookup on a using PRIMARY (articleId=ak1.articleId) (cost=0.25 rows=1) (actual time=0.00522..0.00526 rows=1 loops=5)
|     -> Nested loop inner join (cost=134 rows=1) (actual time=0.00244..0.00244 rows=0 loops=5)
|       -> Constant row from k2 (cost=652e-6..652e-6 rows=1) (actual time=0.00227..0.00227 rows=0 loops=5)
|       -> Single-row covering index lookup on ak2 using PRIMARY (articleId=ak1.articleId, keywordId=NULL) (cost=0.25 rows=1) (never executed)
|
+-----+

1 row in set (0.00 sec)
```

3. Index Articles(articleId) performance degradation (probably because it's redundant and only serves as bulk):


```

-----+
-> Limit: 15 row(s) (actual time=30.1..30.1 rows=15 loops=1)
-> Sort: num_notes DESC, num_keywords DESC, limit input to 15 row(s) per chunk (actual time=30.1..30.1 rows=15 loops=1)
-> Stream results (actual time=23.5..29.9 rows=860 loops=1)
-> Group aggregate: count(distinct FeedNotes.noteId), count(distinct Keywords.keywordId) (actual time=23.4..29.6 rows=860 loops=1)
-> Sort: a.articleId, a.title (actual time=23.3..23.9 rows=6046 loops=1)
-> Stream results (cost=4444 rows=8000) (actual time=0.119..16.3 rows=6046 loops=1)
-> Nested loop inner join (cost=4444 rows=8000) (actual time=0.116..14.1 rows=6046 loops=1)
-> Nested loop inner join (cost=1644 rows=8000) (actual time=0.108..7.91 rows=6046 loops=1)
-> Nested loop inner join (cost=547 rows=1175) (actual time=0.0565..2.73 rows=1282 loops=1)
-> Filter: (f.articleId is not null) (cost=136 rows=1175) (actual time=0.0398..0.541 rows=1282 loops=1)
-> Covering index scan on f using articleId (cost=136 rows=1175) (actual time=0.0322..0.416 rows=1282 loops=1)
-> Single-row index lookup on a using PRIMARY (articleId=f.articleId) (cost=0.25 rows=1) (actual time=0.00149..0.00152 rows=1 loops=1282)
-> Covering index lookup on ak using PRIMARY (articleId=f.articleId) (cost=0.253 rows=6.81) (actual time=0.0024..0.0036 rows=4.72 loops=1282)
-> Single-row covering index lookup on k using PRIMARY (keywordId=ak.keywordId) (cost=0.25 rows=1) (actual time=823e-6..853e-6 rows=1 loops=6046)
|
-----+
row in set (0.04 sec)

mysql> DROP INDEX idx ON Articles;
Query OK, 0 rows affected (0.04 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> CREATE INDEX idx ON Articles(articleId);
Query OK, 0 rows affected (0.33 sec)
Records: 0 Duplicates: 0 Warnings: 0

mysql> EXPLAIN ANALYZE SELECT a.articleId, a.title, COUNT(DISTINCT f.noteId) AS num_notes, COUNT(DISTINCT k.keywordId) AS num_keywords FROM Articles a JOIN FeedNotes f ON a.articleId=f.articleId
DESC, num_keywords DESC LIMIT 15;
-----+
EXPLAIN
-----+
-----+
-> Limit: 15 row(s) (actual time=46.1..46.1 rows=15 loops=1)
-> Sort: num_notes DESC, num_keywords DESC, limit input to 15 row(s) per chunk (actual time=46.1..46.1 rows=15 loops=1)
-> Stream results (actual time=37.45..6 rows=860 loops=1)
-> Group aggregate: count(distinct FeedNotes.noteId), count(distinct Keywords.keywordId) (actual time=36.7..44.8 rows=860 loops=1)
-> Sort: a.articleId, a.title (actual time=36.5..37.6 rows=6046 loops=1)
-> Stream results (cost=4444 rows=8000) (actual time=0.5..20.8 rows=6046 loops=1)
-> Nested loop inner join (cost=4444 rows=8000) (actual time=0.49..18.2 rows=6046 loops=1)
-> Nested loop inner join (cost=1644 rows=8000) (actual time=0.451..10.2 rows=6046 loops=1)

```

Advanced Query #3: -- Articles with keyword Matcha that also have at least 2 keywords (article is content-heavy)

```

SELECT a.title
FROM Articles a
WHERE a.articleId IN (
    SELECT ak1.articleId
    FROM articlesKeywords ak1
    JOIN Keywords k1 ON ak1.keywordId = k1.keywordId
    WHERE k1.keyword = "Australia"
)
AND a.articleId IN (
    SELECT ak2.articleId
    FROM articlesKeywords ak2
    GROUP BY ak2.articleId
    HAVING COUNT(DISTINCT ak2.keywordId) >= 2
)
LIMIT 15;

```

Returns:

```
mysql> SELECT a.title
-> FROM Articles a
-> WHERE a.articleId IN (
->     SELECT ak1.articleId
->     FROM articlesKeywords ak1
->     JOIN Keywords k1 ON ak1.keywordId = k1.keywordId
->     WHERE k1.keyword = "Australia"
-> )
-> AND a.articleId IN (
->     SELECT ak2.articleId
->     FROM articlesKeywords ak2
->     GROUP BY ak2.articleId
->     HAVING COUNT(DISTINCT ak2.keywordId) >= 2
-> )
-> LIMIT 15;

+-----+
| title |
+-----+
| 52 Places to Go in 2025 |
| The Season of the Skulls |
+-----+
2 rows in set (0.01 sec)
```

Advanced queries:
(Before)

```
+-----+
| Limit: 15 row(s) (cost=554 rows=15) (actual time=3.06..5.07 rows=2 loops=1)
-> Nested loop inner join (cost=554 rows=1215) (actual time=3.06..5.07 rows=2 loops=1)
-> Filter: <in_optimizer>(a.articleId,a.articleId in (select #3)) (cost=128 rows=1215) (actual time=2.5..4.01 rows=549 loops=1)
-> Table scan on a (cost=128 rows=1215) (actual time=0.0523..0.507 rows=1266 loops=1)
-> Select #3 (subquery in condition; run only once)
-> Filter: ((a.articleId = <materialized_subquery>'.articleId')) (cost=1155..1155 rows=1) (actual time=0.00253..0.00253 rows=0.433 loops=1267)
-> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=0.00239..0.00239 rows=0.433 loops=1267)
-> Index lookup on <materialized_subquery> using <auto_distinct_key> (articleId=a.articleId) (actual time=0.00227..0.00227 rows=0.433 loops=1267)
-> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=2.42..2.42 rows=549 loops=1)
-> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.028..2.33 rows=549 loops=1)
-> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0266..2.25 rows=860 loops=1)
-> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.0204..1.44 rows=5896 loops=1)
-> Single-row covering index lookup on ak1 using PRIMARY (articleId=a.articleId, keywordId='1577') (cost=0.25 rows=1) (actual time=0.00181..0.00181 rows=0.00364 loops=549)
+-----+
```

1. Index Keywords(keyword) - no improvement

```
+-----+
-> Limit: 15 row(s) (cost=554 rows=15) (actual time=3.45..5.35 rows=2 loops=1)
-> Nested loop inner join (cost=554 rows=1215) (actual time=3.45..5.35 rows=2 loops=1)
-> Filter: <in_optimizer>(a.articleId,a.articleId in (select #3)) (cost=128 rows=1215) (actual time=2.74..4.2 rows=549 loops=1)
-> Table scan on a (cost=128 rows=1215) (actual time=0.0695..0.5 rows=1266 loops=1)
-> Select #3 (subquery in condition; run only once)
-> Filter: ((a.articleId = <materialized_subquery>'.articleId')) (cost=1155..1155 rows=1) (actual time=0.00268..0.00268 rows=0.433 loops=1267)
-> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=0.00254..0.00254 rows=0.433 loops=1267)
-> Index lookup on <materialized_subquery> using <auto_distinct_key> (articleId=a.articleId) (actual time=0.00242..0.00242 rows=0.433 loops=1267)
-> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=2.63..2.63 rows=549 loops=1)
-> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.0279..2.5 rows=549 loops=1)
-> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0264..2.4 rows=860 loops=1)
-> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.0202..1.52 rows=5896 loops=1)
-> Single-row covering index lookup on ak1 using PRIMARY (articleId=a.articleId, keywordId='1577') (cost=0.25 rows=1) (actual time=0.00201..0.00201 rows=0.00364 loops=549)
+-----+
```

2. Index articlesKeywords(keywordId) performance improvement

```

| -> Limit: 15 row(s) (cost=1.75 rows=2) (actual time=2.56..2.57 rows=2 loops=1)
    -> Nested loop inner join (cost=1.75 rows=2) (actual time=2.56..2.57 rows=2 loops=1)
        -> Filter: <in_optimizer>(ak1.articleId,ak1.articleId in (select #3)) (cost=1.05 rows=2) (actual time=2.54..2.55 rows=2 loops=1)
            -> Covering index lookup on ak1 using idx (keywordId='1577') (cost=1.05 rows=2) (actual time=0.0119..0.0157 rows=2 loops=1)
            -> Select #3 (subquery in condition; run only once)
                -> Filter: ((ak1.articleId = '<materialized_subquery>'.articleId)) (cost=1155..1155 rows=1) (actual time=0.841..0.841 rows=0.667 loops=3)
                    -> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=0.84..0.84 rows=0.667 loops=3)
                        -> Index lookup on <materialized_subquery> using <auto_distinct_key> (articleId=ak1.articleId) (actual time=0.84..0.84 rows=0.667 loops=3)
                            -> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=2.51..2.51 rows=549 loops=1)
                                -> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.0314..2.42 rows=549 loops=1)
                                    -> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0299..2.33 rows=860 loops=1)
                                        -> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.0238..1.42 rows=5896 loops=1)
                                            -> Single-row index lookup on a using PRIMARY (articleId=ak1.articleId) (cost=0.3 rows=1) (actual time=0.01..0.0101 rows=1 loops=2)

```

3. Index Articles(abstract) no improvement

```

| -> Limit: 15 row(s) (cost=554 rows=15) (actual time=4.71..7 rows=2 loops=1)
    -> Nested loop inner join (cost=554 rows=1215) (actual time=4.71..7 rows=2 loops=1)
        -> Filter: <in_optimizer>(a.articleId,a.articleId in (select #3)) (cost=128 rows=1215) (actual time=3.94..5.79 rows=549 loops=1)
            -> Table scan on a (cost=128 rows=1215) (actual time=0.077..0.688 rows=1266 loops=1)
            -> Select #3 (subquery in condition; run only once)
                -> Filter: ((a.articleId = '<materialized_subquery>'.articleId)) (cost=1155..1155 rows=1) (actual time=0.00375..0.00375 rows=0.433 loops=1267)
                    -> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=0.00361..0.00361 rows=0.433 loops=1267)
                        -> Index lookup on <materialized_subquery> using <auto_distinct_key> (articleId=a.articleId) (actual time=0.00348..0.00348 rows=0.433 loops=1267)
                            -> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=3.82..3.82 rows=549 loops=1)
                                -> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.0491..3.63 rows=549 loops=1)
                                    -> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0472..3.48 rows=860 loops=1)
                                        -> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.0379..1.9 rows=5896 loops=1)
                                            -> Single-row covering index lookup on ak1 using PRIMARY (articleId=a.articleId, keywordId='1577') (cost=0.25 rows=1) (actual time=0.00209..0.00209 rows=0.00364 loops=549)

```

Advanced Query #4: -- Keywords that are only found in one article

```

SELECT k.keyword
FROM Keywords k
JOIN articlesKeywords ak ON k.keywordId = ak.keywordId
GROUP BY k.keywordId, k.keyword
HAVING COUNT(DISTINCT ak.articleId) = 5
LIMIT 15;

```

Returns:

```
mysql> SELECT k.keyword
-> FROM Keywords k
-> JOIN articlesKeywords ak ON k.keywordId = ak.keywordId
-> GROUP BY k.keywordId, k.keyword
-> HAVING COUNT(DISTINCT ak.articleId) = 5
-> LIMIT 15;
```

```
+-----+
| keyword |
+-----+
| Gardens and Gardening |
| Holden, Stephen |
| Jagger, Mick |
| Germany |
| Economic Conditions and Trends |
| Budgets and Budgeting |
| Families and Family Life |
| Hate Crimes |
| Men and Boys |
| Local Government |
| Washington (DC) |
| Federal Budget (US) |
| Theater (Off Broadway) |
| Archives and Records |
| Israeli Settlements |
+-----+
15 rows in set (0.03 sec)
```

Advanced Queries:
(Before)

```
+-----+
| -> Limit: 15 row(s) (cost=554 rows=15) (actual time=3.1..5.5 rows=2 loops=1)
-> Nested loop inner join (cost=554 rows=1215) (actual time=3.1..5.5 rows=2 loops=1)
-> Filter: <in_optimizer>(a.articleId,a.articleId in (select #3)) (cost=128 rows=1215) (actual time=2.44..4.26 rows=549 loops=1)
-> Table scan on a (cost=128 rows=1215) (actual time=0.0381..0.57 rows=1266 loops=1)
-> Select #3 (subquery in condition; run only once)
-> Filter: ((a.articleId = <materialized_subquery>'.articleId)) (cost=1155..1155 rows=1) (actual time=0.00262..0.00262 rows=0.433 loops=1267)
-> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=0.00244..0.00244 rows=0.433 loops=1267)
-> Index lookup on <materialized_subquery> using <auto_distinct_key> (articleId=a.articleId) (actual time=0.0023..0.0023 rows=0.433 loops=1267)
-> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=2.36..2.36 rows=549 loops=1)
-> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.031..2.27 rows=549 loops=1)
-> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0294..2.18 rows=860 loops=1)
-> Select #3 (subquery in condition; run only once)
-> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.0226..1.36 rows=5896 loops=1)
-> Single-row covering index lookup on ak1 using PRIMARY (articleId=a.articleId, keywordId='1577') (cost=0.25 rows=1) (actual time=0.00213..0.00213 rows=0.00364 loops=549)
+-----+
```

1. Index articlesKeywords(keywordId) - performance improvement

```
+-----+
| -> Limit: 15 row(s) (cost=1.75 rows=2) (actual time=3.13..3.15 rows=2 loops=1)
-> Nested loop inner join (cost=1.75 rows=2) (actual time=3.13..3.15 rows=2 loops=1)
-> Filter: <in_optimizer>(ak1.articleId,ak1.articleId in (select #3)) (cost=1.05 rows=2) (actual time=3.07..3.08 rows=2 loops=1)
-> Covering index lookup on ak1 using idx (keywordId='1577') (cost=1.05 rows=2) (actual time=0.0341..0.0387 rows=2 loops=1)
-> Select #3 (subquery in condition; run only once)
-> Filter: ((ak1.articleId = <materialized_subquery>'.articleId)) (cost=1155..1155 rows=1) (actual time=1.01..1.01 rows=0.667 loops=3)
-> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=1.01..1.01 rows=0.667 loops=3)
-> Index lookup on <materialized_subquery> using <auto_distinct_key> (articleId=ak1.articleId) (actual time=1..1 rows=0.667 loops=3)
-> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=3..3 rows=549 loops=1)
-> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.0381..2.82 rows=549 loops=1)
-> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0354..2.64 rows=860 loops=1)
-> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.0274..1.66 rows=5896 loops=1)
-> Single-row index lookup on a using PRIMARY (articleId=ak1.articleId) (cost=0.3 rows=1) (actual time=0.034..0.0341 rows=1 loops=2)
+-----+
```

2. Index Keywords(keywordId) - no improvement

```

| -> Limit: 15 row(s) (cost=554 rows=15) (actual time=3.65..6.21 rows=2 loops=1)
    -> Nested loop inner join (cost=554 rows=1215) (actual time=3.65..6.21 rows=2 loops=1)
        -> Filter: <in_optimizer>(a.articleId,a.articleId in (select #3)) (cost=128 rows=1215) (actual time=3..4.9 rows=549 loops=1)
            -> Table scan on a (cost=128 rows=1215) (actual time=0.0628..0.637 rows=1266 loops=1)
            -> Select #3 (subquery in condition: run only once)
                -> Filter: ((a.articleId = '<materialized_subquery>'.articleId)) (cost=1155..1155 rows=1) (actual time=0.00305..0.00305 rows=0.433 loops=1267)
                    -> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=0.00287..0.00287 rows=0.433 loops=1267)
                        -> Index lookup on <materialized_subquery> using <auto distinct key> (articleId=a.articleId) (actual time=0.00272..0.00272 rows=0.433 loops=1267)
                            -> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=2.89..2.89 rows=549 loops=1)
                                -> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.0314..2.77 rows=549 loops=1)
                                    -> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0299..2.66 rows=860 loops=1)
                                        -> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.023..1.62 rows=5896 loops=1)
                                            -> Single-row covering index lookup on ak1 using PRIMARY (articleId=a.articleId, keywordId='1577') (cost=0.25 rows=1) (actual time=0.00224..0.00224 rows=0.00364 loops=549)

```

3. Index Keywords(keyword) - no improvement

```

| -> Limit: 15 row(s) (cost=554 rows=15) (actual time=3.39..5.32 rows=2 loops=1)
    -> Nested loop inner join (cost=554 rows=1215) (actual time=3.39..5.32 rows=2 loops=1)
        -> Filter: <in_optimizer>(a.articleId,a.articleId in (select #3)) (cost=128 rows=1215) (actual time=2.76..4.3 rows=549 loops=1)
            -> Table scan on a (cost=128 rows=1215) (actual time=0.127..0.634 rows=1266 loops=1)
            -> Select #3 (subquery in condition: run only once)
                -> Filter: ((a.articleId = '<materialized_subquery>'.articleId)) (cost=1155..1155 rows=1) (actual time=0.00265..0.00265 rows=0.433 loops=1267)
                    -> Limit: 1 row(s) (cost=1155..1155 rows=1) (actual time=0.00252..0.00252 rows=0.433 loops=1267)
                        -> Index lookup on <materialized_subquery> using <auto distinct key> (articleId=a.articleId) (actual time=0.00239..0.00239 rows=0.433 loops=1267)
                            -> Materialize with deduplication (cost=1155..1155 rows=788) (actual time=2.6..2.6 rows=549 loops=1)
                                -> Filter: (count(distinct ak2.keywordId) >= 5) (cost=1076 rows=788) (actual time=0.0516..2.48 rows=549 loops=1)
                                    -> Group aggregate: count(distinct ak2.keywordId) (cost=1076 rows=788) (actual time=0.0495..2.39 rows=860 loops=1)
                                        -> Covering index scan on ak2 using PRIMARY (cost=540 rows=5365) (actual time=0.0235..1.5 rows=5896 loops=1)
                                            -> Single-row covering index lookup on ak1 using PRIMARY (articleId=a.articleId, keywordId='1577') (cost=0.25 rows=1) (actual time=0.00176..0.00176 rows=0.00364 loops=549)

```

We will be going with the “enhanced composite key” index design. We will apply an index to the second element in the articlesKeywords composite key, because by default the index for this table will be the first element in the composite key, which is articleId. So having the secondary key of the composite key be used as an index speeds up the JOIN and related lookups, hence the various performance improvements throughout our advanced query index testing as shown above.